

RAPPORT

Projet llm-devops



DÉPLOIEMENT D'UN LARGE LANGUAGE MODEL OPEN-SOURCE

Docker Compose et Kubernetes

01

Introduction

Les modèles de langage open-source (LLM) connaissent une adoption croissante, portée par la volonté de nombreuses organisations de garder la main sur leurs données plutôt que de dépendre systématiquement d'API propriétaires externes. Des outils comme Ollama rendent aujourd'hui possible l'exécution locale de ces modèles, à condition de disposer d'une infrastructure capable de les héberger de façon fiable : un LLM est une charge de travail exigeante, gourmande en mémoire, avec des temps de démarrage longs et des besoins de stockage conséquents pour les poids des modèles.

Faire tourner un tel service sur une seule machine, via un simple conteneur, est relativement direct. La question devient plus intéressante dès que l'on cherche à en garantir la disponibilité, la résilience face aux pannes, et la sécurité des échanges réseau entre les composants : comment déployer une application conteneurisée, aussi lourde qu'un LLM, de manière à ce qu'elle survive à un redémarrage, conserve ses données, et reste accessible même en cas de défaillance partielle de l'infrastructure ? C'est cette problématique de fiabilisation d'un déploiement, au-delà du simple fait de le faire fonctionner, qui a guidé ce projet.



À propos

L'application étudiée repose sur deux services complémentaires : Ollama, qui héberge et exécute les modèles de langage, et Open WebUI, une interface web permettant d'interagir avec ces modèles. Elle a d'abord été déployée avec Docker Compose, pour en comprendre l'architecture dans un cadre simple, avant d'être entièrement migrée vers Kubernetes, seul environnement des deux à offrir nativement les mécanismes nécessaires pour répondre à la problématique posée : redémarrage automatique des composants défaillants, stockage persistant découplé du cycle de vie des conteneurs, supervision continue de l'état de santé, et isolation réseau entre services.

Ce projet, et le présent rapport, retracent cette démarche : les choix techniques effectués à chaque étape, les incidents réels rencontrés en cours de déploiement, ainsi que les mécanismes supplémentaires mis en place pour renforcer la résilience et la sécurité de l'ensemble.

02

Déploiement local avec Docker Compose

La première étape a consisté à déployer l'application avec le fichier `docker-compose.yml` fourni, qui définit deux services : `ollama` et `open-webui`, reliés par un réseau bridge dédié (`llm`) et disposant chacun d'un volume nommé pour la persistance des données.

Le déploiement a été validé en plusieurs temps : vérification de l'état des conteneurs avec `docker compose ps`, confirmant que les deux services étaient démarrés et que le healthcheck d'Ollama passait au vert ; inspection des logs pour écarter toute erreur de démarrage ; puis test fonctionnel confirmant que les deux applications répondaient correctement sur leurs ports respectifs (11434 pour Ollama, 3000 pour Open WebUI).

Un ajustement a été nécessaire sur le healthcheck d'Open WebUI : la commande fournie reposait sur `wget`, absent de l'image officielle, ce qui provoquait un faux statut « unhealthy » alors que l'application répondait normalement (vérifié via `curl`).

Trois modèles ont ensuite été téléchargés dans Ollama (`llama3.2:3b`, `mistral:7b` et `deepseek-r1:8b`) et testés individuellement depuis l'interface Open WebUI, chacun ayant généré une réponse cohérente à un prompt donné.

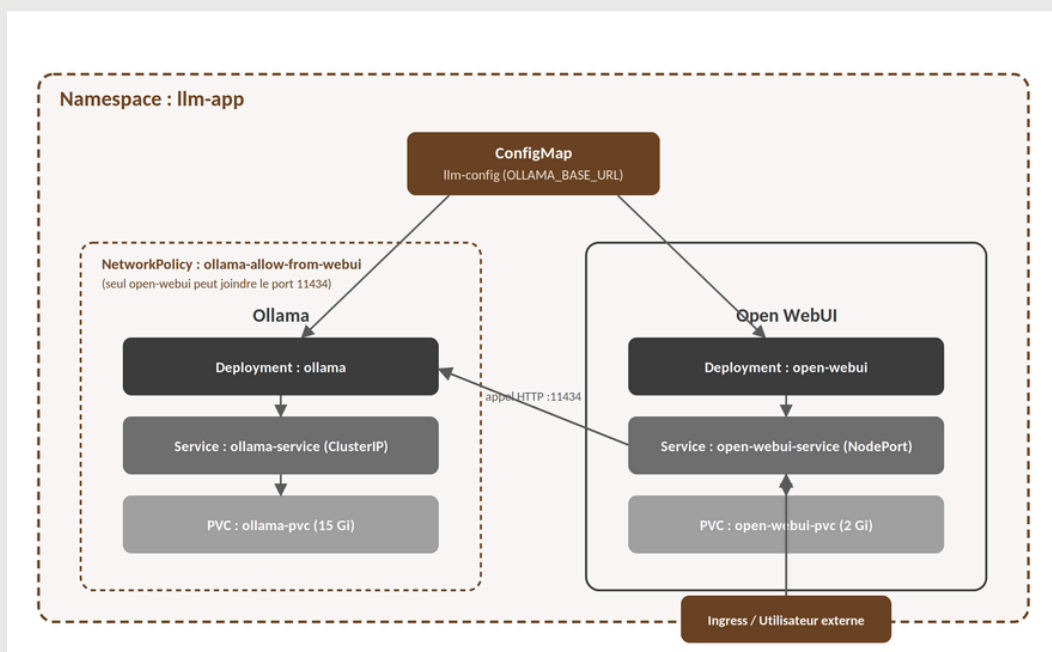
03

Migration vers Kubernetes

La seconde partie du projet a consisté à recréer entièrement l'architecture Docker Compose sous forme de manifests Kubernetes, déployés sur un cluster local Minikube.

Vue d'ensemble de l'architecture

L'ensemble des ressources a été regroupé dans un namespace dédié (llm-app) afin d'isoler le projet du reste du cluster. Chaque service applicatif (Ollama, Open WebUI) est géré par un Deployment associé à un Service et à un PersistentVolumeClaim, le tout relié par un ConfigMap partagé et sécurisé par des NetworkPolicy.



Ressource	Rôle dans le projet
Namespace (llm-app)	Isole toutes les ressources du projet dans un espace logique dédié, distinct du namespace default.
ConfigMap (llm-config)	Centralise la configuration applicative (URL d'Ollama, nom de l'interface) sans coder ces valeurs en dur dans les manifests.
PersistentVolumeClaim	Réserve un espace de stockage durable pour Ollama (modèles téléchargés) et Open WebUI (comptes, conversations).
Deployment (Ollama / Open WebUI)	Gère le cycle de vie des pods : création, redémarrage automatique en cas d'échec, mise à jour progressive.
Service (ClusterIP / NodePort)	Fournit une adresse réseau stable pour joindre les pods, indépendamment de leur IP interne qui change à chaque redémarrage.
Ingress	Point d'entrée HTTP unique et nommé pour accéder à l'application depuis l'extérieur du cluster.
NetworkPolicy	Restreint les flux réseau autorisés entre les pods, selon le principe du moindre privilège.
PodDisruptionBudget	Garantit un nombre minimal de pods disponibles pendant une opération de maintenance planifiée du cluster.

NB : l'accès à l'application via l'Ingress (nom d'hôte llm.local) s'est révélé impossible depuis Windows avec le driver Docker de Minikube, l'adresse IP interne du cluster n'étant pas routable depuis l'hôte. Conformément à l'alternative explicitement prévue dans les consignes du projet, l'accès à l'application pour les tests et les captures d'écran a été réalisé via kubectl port-forward, une solution robuste et suffisante pour un environnement de développement local. L'Ingress reste néanmoins déployé et fonctionnel en tant que ressource du cluster.

Communication entre les services

Contrairement à Docker Compose, où les conteneurs se joignent par leur nom de service sur un réseau bridge commun, Kubernetes s'appuie sur le DNS interne du cluster : chaque Service expose un nom stable (par exemple ollama-service) résolu automatiquement par CoreDNS vers l'adresse IP courante des pods correspondants, même si ces derniers sont recréés et changent d'adresse IP.

Concrètement, Open WebUI contacte Ollama via l'URL `http://ollama-service:11434`, injectée automatiquement dans le conteneur grâce à la variable `OLLAMA_BASE_URL` définie dans le ConfigMap. Le Service `ollama-service`, de type `ClusterIP`, n'est accessible que depuis l'intérieur du cluster : Ollama n'a pas besoin d'être exposé publiquement, seule l'interface Open WebUI doit l'être. Cette dernière est exposée via un Service de type `NodePort` (port 30080) et un objet Ingress, ce qui permet d'y accéder depuis l'extérieur du cluster.

Stockage persistant

Deux `PersistentVolumeClaim` ont été créés : `ollama-pvc` pour les modèles téléchargés, et `open-webui-pvc` pour les comptes utilisateurs et l'historique des conversations. Sans ce stockage persistant, chaque redémarrage d'un pod entraînerait la perte complète des modèles téléchargés ou des données de l'interface, puisque le système de fichiers d'un conteneur est par défaut éphémère.

Une limite du provisioner de stockage par défaut de Minikube (`k8s.io/minikube-hostpath`) a également été observée : bien que le PVC déclare une capacité de 15 Gi, l'espace réellement utilisable sur le nœud dépasse largement cette valeur, car ce provisioner ne fait pas de véritable application de quota (contrairement à un `StorageClass` cloud comme AWS EBS). Ce même provisioner ne permet pas non plus l'extension à chaud d'un volume existant (`ALLOWVOLUMEEXPANSION` à `false`), ce qui a orienté certaines décisions prises en cours de projet.

Sondes de santé (probes)

Les trois types de sondes demandées ont été implémentées sur les deux Deployment :

- Sonde de démarrage (startupProbe) : laisse le temps au conteneur de démarrer complètement avant que les autres sondes ne commencent à le surveiller, évitant un redémarrage prématuré pendant un chargement long.
- Sonde de préparation (readinessProbe) : détermine si le pod est prêt à recevoir du trafic. En cas d'échec, le pod est simplement retiré du Service, sans être redémarré.
- Sonde de vivacité (livenessProbe) : détermine si le pod est toujours fonctionnel. En cas d'échecs répétés, Kubernetes redémarre automatiquement le conteneur.

Le conteneur Ollama utilise une vérification de type tcpSocket sur le port 11434 (le point de terminaison racine de l'API ne renvoie pas de code HTTP simple à interpréter), tandis qu'Open WebUI utilise une vérification httpGet sur l'endpoint /health, qui renvoie une véritable réponse applicative, corrigeant au passage le souci de healthcheck rencontré en Docker Compose.

Preuve de fonctionnement : le mécanisme de liveness a été observé en conditions réelles lors du projet. Le pod Ollama, ayant dépassé sa limite mémoire pendant la génération d'une réponse par le modèle deepseek-r1:8b, a été automatiquement arrêté (OOMKilled) puis relancé par Kubernetes sans intervention manuelle, illustrant concrètement l'intérêt de ces sondes pour la résilience de l'application.

Gestion de la configuration

Un ConfigMap unique (llm-config) centralise les variables de configuration partagées, notamment OLLAMA_BASE_URL. Il est injecté dans le Deployment Open WebUI via envFrom, ce qui évite de coder ces valeurs en dur dans les manifests. Le bon fonctionnement de ce mécanisme a été vérifié visuellement : le titre affiché dans l'interface (« Local LLM Chat ») correspond exactement à la valeur WEBUI_NAME définie dans le ConfigMap.

Docker Compose et Kubernetes : quelles différences ?

Si les deux outils permettent d'orchestrer des applications conteneurisées, ils répondent à des besoins différents. Docker Compose reste adapté à un usage local, simple et rapide à mettre en place, tandis que Kubernetes introduit une couche d'abstraction supplémentaire (nœuds, pods, contrôleurs) pensée pour la production, la haute disponibilité et la montée en charge.

Aspect	Docker Compose	Kubernetes
Portée	Une seule machine locale	Un cluster de un ou plusieurs nœuds
Résilience	Redémarre un conteneur arrêté (restart: unless-stopped)	Redémarre automatiquement un pod défaillant et peut le replanifier sur un autre nœud
Mise à l'échelle	Manuelle, limitée à la machine hôte	Déclarative, peut répartir la charge sur plusieurs nœuds
Découverte de service	Nom du service dans le réseau bridge Compose	DNS interne du cluster via les objets Service
Configuration	Fichier .env et variables d'environnement	ConfigMap et Secret, injectables dans plusieurs déploiements
Supervision de santé	healthcheck basique (une seule sonde)	Trois sondes distinctes : startup, readiness, liveness
Cas d'usage	Développement local, prototypage rapide	Environnements de production nécessitant scalabilité et haute disponibilité

05 Autres fonctionnalités implémentées

NetworkPolicy

1

Par défaut, tous les pods d'un cluster Kubernetes peuvent communiquer librement entre eux, ce qui ne respecte pas le principe du moindre privilège. Deux NetworkPolicy ont été créées : la première autorise le trafic entrant vers le pod Ollama uniquement depuis les pods portant le label `app: open-webui`, sur le port 11434 ; la seconde autorise tout trafic entrant vers Open WebUI, ce dernier devant rester accessible depuis l'extérieur du cluster.

Une limite a été identifiée et documentée : le CNI par défaut de Minikube (bridge) ne supporte pas l'application effective des NetworkPolicy. Les ressources sont correctement créées et acceptées par l'API Kubernetes, mais le trafic n'est pas réellement filtré dans cet environnement de développement. En production, avec un CNI compatible tel que Calico ou Cilium, cette règle serait activement appliquée par le plugin réseau.

PodDisruptionBudget

2

Un PodDisruptionBudget (`minAvailable: 1`) a été défini pour chacun des deux Deployments, garantissant qu'au moins un pod reste disponible lors d'une interruption volontaire planifiée du cluster (par exemple, la maintenance d'un nœud). Avec un seul replica par Deployment, la valeur `ALLOWED DISRUPTIONS` observée est de 0 : Kubernetes refuse toute interruption volontaire tant qu'aucun second replica ne peut prendre le relais. Cette observation illustre bien la limite d'une haute disponibilité reposant sur un seul replica, et la nécessité d'un scaling horizontal pour une résilience réelle en production.

Pipeline GitHub Actions

3

Un workflow de validation continue a été mis en place, déclenché automatiquement à chaque push ou pull request vers la branche principale du dépôt. Il installe l'outil `kubeconform` pour valider hors ligne la syntaxe et la conformité au schéma Kubernetes de chaque manifest, sans nécessiter de cluster réel accessible depuis l'environnement d'exécution GitHub Actions, puis vérifie la validité du fichier `docker-compose.yml`. Ce pipeline garantit qu'aucun manifest syntaxiquement incorrect ne peut être fusionné sans être détecté.

Conclusion

Ce projet répond à la problématique posée en introduction : comment déployer une application aussi exigeante qu'un LLM de manière à ce qu'elle survive à un redémarrage, conserve ses données et reste accessible malgré une défaillance partielle de l'infrastructure. L'application a d'abord été déployée avec Docker Compose, ce qui a permis d'en comprendre l'architecture dans un cadre simple, puis entièrement migrée vers Kubernetes, seul environnement à offrir nativement les mécanismes attendus : redémarrage automatique des pods défaillants via les Deployment, stockage persistant découplé du cycle de vie des conteneurs via les PersistentVolumeClaim, supervision continue de l'état de santé via les trois niveaux de sondes, et configuration centralisée via un ConfigMap partagé.

L'ensemble des ressources obligatoires a été mis en place et validé sur un cluster Minikube : namespace dédié, Deployment et Service pour chacun des deux composants, PersistentVolumeClaim, ConfigMap et Ingress. Les critères de validation ont tous été vérifiés avec succès, avec les trois modèles de langage requis téléchargés, sélectionnables et fonctionnels depuis l'interface Open WebUI. Le déploiement a en outre été renforcé par trois fonctionnalités bonus complémentaires : des NetworkPolicy restreignant les flux réseau entre les pods, un PodDisruptionBudget garantissant une disponibilité minimale lors des opérations de maintenance, et un pipeline GitHub Actions validant automatiquement l'ensemble des manifests à chaque modification du dépôt.

Ce projet a permis de consolider une compréhension pratique de l'orchestration de conteneurs, de la simple exécution locale d'une application jusqu'à un déploiement structuré, résilient et sécurisé, proche des standards attendus en environnement de production.